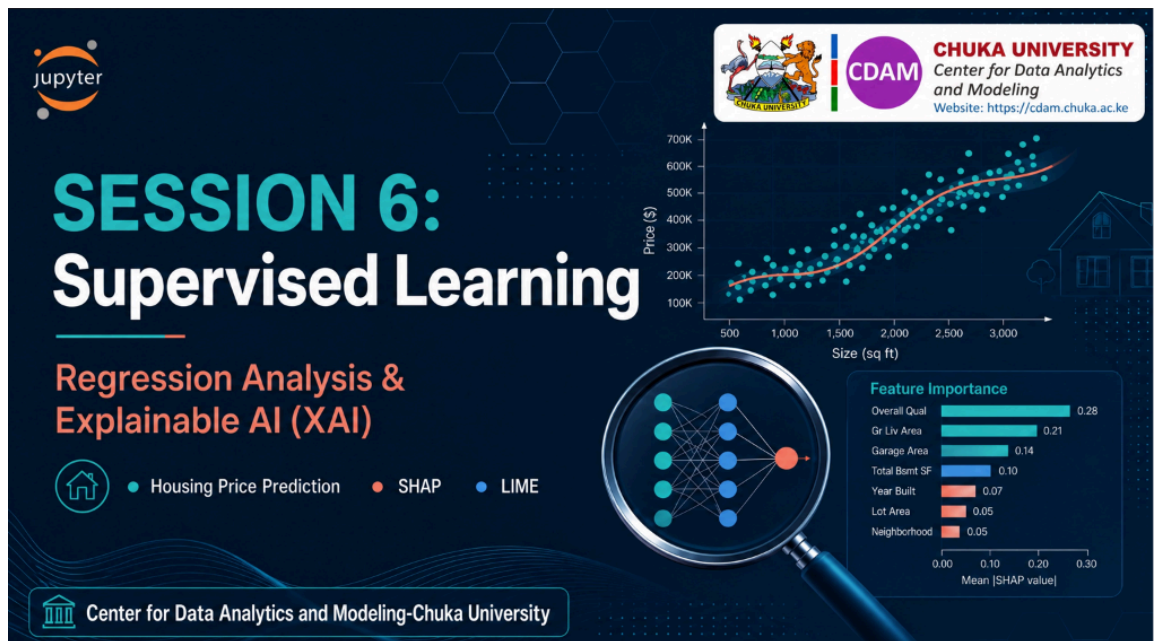




Introduction to Housing Price Prediction

Housing price prediction is one of the most widely studied applications of **regression analysis** and **machine learning**. The objective is to estimate the market value of a house based on its characteristics, such as location, size, number of rooms, population density, median income, and other socioeconomic or environmental factors. Accurate house price prediction benefits home buyers, sellers, real estate investors, financial institutions, and policymakers by supporting informed decision-making and fair property valuation.

Traditional statistical methods such as **Linear Regression** have long been used to model the relationship between housing prices and explanatory variables. However, housing markets often exhibit complex, nonlinear relationships that cannot be fully captured by simple linear models. As a result, modern machine learning algorithms such as **k-Nearest Neighbors (k-NN)**, **Decision Trees**, **Random Forests**, and **Polynomial Regression** have become popular due to their ability to learn intricate patterns from data and produce more accurate predictions.

In this practical session, the **California Housing Dataset**, an in-built dataset available in Scikit-learn, is used to demonstrate the complete workflow of a regression-based machine learning project. The dataset contains information collected from the 1990 U.S. Census for districts in California. The target variable is the **median house value**, while the predictor variables describe demographic, geographic, and economic characteristics of each district.

The analysis follows a complete machine learning pipeline, including:

- Loading and exploring the housing dataset.
- Splitting the data into training and testing sets.

- Standardizing predictor variables.
- Building multiple regression models.
- Evaluating model performance using **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, **Mean Absolute Error (MAE)**, and the **Coefficient of Determination (R^2)**.
- Investigating the **bias-variance tradeoff** through learning curves.
- Performing **feature selection** to identify the most influential predictors.
- Optimizing model performance using **GridSearchCV** and **RandomizedSearchCV**.
- Applying **Explainable Artificial Intelligence (XAI)** techniques, including **SHAP** and **LIME**, to interpret both global and local model predictions.

By the end of this exercise, learners will understand how different regression algorithms perform on a real-world housing dataset, how to improve predictive performance through model optimization, and how to interpret machine learning models using modern explainability techniques. This provides a comprehensive introduction to predictive analytics for continuous outcomes using Python and Scikit-learn.

STANDARD OLS APPROACH FOR REGRESSION ANALYSIS (Recap from Introductory)

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Core Libraries
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Statsmodels: The gold standard for OLS
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, acorr_ljungbox
from scipy import stats

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

```
In [2]: # Load California Housing dataset
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal')

# Subsample for faster execution
```

```

X = X.sample(n=3000, random_state=RANDOM_STATE).reset_index(drop=True)
y = y.loc[X.index].reset_index(drop=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)

```

```

In [3]: print(f"Training samples: {X_train.shape[0]}")
print(f"Test samples:      {X_test.shape[0]}")
print(f"Features:          {X_train.shape[1]}")
print(f"\nFeature Descriptions:")
feature_desc = {
    'MedInc': 'Median income in block group',
    'HouseAge': 'Median house age in block group',
    'AveRooms': 'Average number of rooms per household',
    'AveBedrms': 'Average number of bedrooms per household',
    'Population': 'Block group population',
    'AveOccup': 'Average number of household members',
    'Latitude': 'Block group latitude',
    'Longitude': 'Block group longitude'
}
for feat, desc in feature_desc.items():
    print(f" {feat:<12} : {desc}")

```

Training samples: 2400

Test samples: 600

Features: 8

Feature Descriptions:

```

MedInc      : Median income in block group
HouseAge    : Median house age in block group
AveRooms    : Average number of rooms per household
AveBedrms   : Average number of bedrooms per household
Population  : Block group population
AveOccup    : Average number of household members
Latitude    : Block group latitude
Longitude   : Block group longitude

```

STATSMODELS OLS (The Statistical Approach)

```

In [4]: # statsmodels requires explicit intercept (constant) column
X_train_sm = sm.add_constant(X_train) # Adds 'const' column of 1s
X_test_sm = sm.add_constant(X_test)

# Fit OLS model
ols_model = sm.OLS(y_train, X_train_sm).fit()

# --- FULL OLS SUMMARY ---
print("\n" + "-" * 80)
print(ols_model.summary())
print("-" * 80)

```

```

-----
                        OLS Regression Results
=====
Dep. Variable:          MedHouseVal    R-squared:                0.002
Model:                  OLS            Adj. R-squared:          -0.001
Method:                 Least Squares   F-statistic:             0.6038
Date:                   Tue, 07 Jul 2026 Prob (F-statistic):       0.775
Time:                   14:22:21        Log-Likelihood:          -3290.8
No. Observations:       2400           AIC:                     6600.
Df Residuals:           2391           BIC:                     6652.
Df Model:               8
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	1.4045	2.665	0.527	0.598	-3.821	6.630
MedInc	-0.0139	0.017	-0.837	0.403	-0.047	0.019
HouseAge	0.0004	0.002	0.234	0.815	-0.003	0.004
AveRooms	0.0212	0.024	0.891	0.373	-0.025	0.068
AveBedrms	-0.0383	0.118	-0.323	0.747	-0.270	0.194
Population	1.816e-05	1.98e-05	0.917	0.359	-2.07e-05	5.7e-05
AveOccup	-0.0126	0.009	-1.395	0.163	-0.030	0.005
Latitude	-0.0144	0.029	-0.500	0.617	-0.071	0.042
Longitude	-0.0053	0.030	-0.176	0.860	-0.065	0.054

```

=====
Omnibus:                523.186    Durbin-Watson:            2.077
Prob(Omnibus):           0.000    Jarque-Bera (JB):         987.106
Skew:                    1.330    Prob(JB):                 4.49e-215
Kurtosis:                 4.672    Cond. No.:                 2.43e+05
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.43e+05. This might indicate that there are strong multicollinearity or other numerical problems.

```

In [5]: print("\n--- Key OLS Metrics (Extracted) ---")
print(f"R-squared:          {ols_model.rsquared:.6f}")
print(f"Adjusted R-squared: {ols_model.rsquared_adj:.6f}")
print(f"F-statistic:          {ols_model.fvalue:.4f}")
print(f"F-test p-value:       {ols_model.f_pvalue:.2e}")
print(f"AIC:                 {ols_model.aic:.4f}")
print(f"BIC:                 {ols_model.bic:.4f}")
print(f"Log-Likelihood:        {ols_model.llf:.4f}")
#print(f"Durbin-Watson:    {ols_model.durbin_watson:.4f}")
print(f" (DW ≈ 2 means no autocorrelation; <1 or >3 is concerning)")

```

```

--- Key OLS Metrics (Extracted) ---
R-squared:          0.002016
Adjusted R-squared: -0.001323
F-statistic:        0.6038
F-test p-value:     7.75e-01
AIC:                6599.5804
BIC:                6651.6294
Log-Likelihood:     -3290.7902
(DW ≈ 2 means no autocorrelation; <1 or >3 is concerning)

```

COEFFICIENT TABLES (Detailed Interpretation)

```

In [6]: # --- Table 1: Full Coefficient Statistics ---
coef_table = ols_model.summary2().tables[1]
print("\n--- Full Coefficient Table (from statsmodels) ---")
print(coef_table.to_string())

--- Full Coefficient Table (from statsmodels) ---

```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
const	1.404535	2.664710	0.527087	0.598182	-3.820847	6.629916
MedInc	-0.013914	0.016632	-0.836613	0.402894	-0.046529	0.018700
HouseAge	0.000418	0.001784	0.234451	0.814655	-0.003080	0.003917
AveRooms	0.021167	0.023754	0.891115	0.372957	-0.025413	0.067747
AveBedrms	-0.038257	0.118392	-0.323135	0.746622	-0.270419	0.193906
Population	0.000018	0.000020	0.917267	0.359095	-0.000021	0.000057
AveOccup	-0.012600	0.009031	-1.395134	0.163105	-0.030310	0.005110
Latitude	-0.014364	0.028726	-0.500016	0.617110	-0.070695	0.041967
Longitude	-0.005341	0.030363	-0.175895	0.860391	-0.064881	0.054199

```

In [7]: # --- Table 2: Cleaned and Formatted ---
print("\n\n--- Cleaned Coefficient Table ---")
clean_table = pd.DataFrame({
    'Feature':      ols_model.params.index,
    'Coefficient':  ols_model.params.values,
    'Std Error':    ols_model.bse.values,
    't-statistic':  ols_model.tvalues.values,
    'p-value':      ols_model.pvalues.values,
    'CI Lower 95%': ols_model.conf_int()[0].values,
    'CI Upper 95%': ols_model.conf_int()[1].values,
    'Significant?': ['Yes' if p < 0.05 else 'No' for p in ols_model.pvalues.values]
})
# Format for display
print(clean_table.to_string(index=False, float_format=lambda x: f'{x:>12.6f}'))

```

```

--- Cleaned Coefficient Table ---
Feature Coefficient Std Error t-statistic p-value CI Lower 95% CI Upper 95% Significant?
const 1.404535 2.664710 0.527087 0.598182 -3.820847 6.629916 No
MedInc -0.013914 0.016632 -0.836613 0.402894 -0.046529 0.018700 No
HouseAge 0.000418 0.001784 0.234451 0.814655 -0.003080 0.003917 No
AveRooms 0.021167 0.023754 0.891115 0.372957 -0.025413 0.067747 No
AveBedrms -0.038257 0.118392 -0.323135 0.746622 -0.270419 0.193906 No
Population 0.000018 0.000020 0.917267 0.359095 -0.000021 0.000057 No
AveOccup -0.012600 0.009031 -1.395134 0.163105 -0.030310 0.005110 No
Latitude -0.014364 0.028726 -0.500016 0.617110 -0.070695 0.041967 No
Longitude -0.005341 0.030363 -0.175895 0.860391 -0.064881 0.054199 No

```

```

In [8]: # --- Table 3: Interpretation Guide ---
print("\n\n--- Coefficient Interpretation (Sorted by Absolute Value) ---")
interpretation = clean_table[clean_table['Feature'] != 'const'].copy()
interpretation['Abs_Coeff'] = interpretation['Coefficient'].abs()
interpretation = interpretation.sort_values('Abs_Coeff', ascending=False)

print(f"\n{'Feature':<12} | {'Coefficient':>12} | {'Direction':>10} | {'Significance':>10}")
print("-" * 95)
for _, row in interpretation.iterrows():
    feat = row['Feature']
    coef = row['Coefficient']
    direction = 'Positive ↑' if coef > 0 else 'Negative ↓'
    sig = '*** p<0.001' if row['p-value'] < 0.001 else ('** p<0.01' if row['p-value'] < 0.01 else 'p > 0.05')
    interp = f"1-unit increase in {feat} → {abs(coef):.4f} {'increase' if coef > 0 else 'decrease'}"
    print(f"{feat:<12} | {coef:>12.6f} | {direction:>10} | {sig:>12} | {interp}")

```

--- Coefficient Interpretation (Sorted by Absolute Value) ---

Feature	Coefficient	Direction	Significance	Interpretation
AveBedrms	-0.038257	Negative ↓	Not Sig	1-unit increase in AveBedrms → 0.0383 decrease in house value
AveRooms	0.021167	Positive ↑	Not Sig	1-unit increase in AveRooms → 0.0212 increase in house value
Latitude	-0.014364	Negative ↓	Not Sig	1-unit increase in Latitude → 0.0144 decrease in house value
MedInc	-0.013914	Negative ↓	Not Sig	1-unit increase in MedInc → 0.0139 decrease in house value
AveOccup	-0.012600	Negative ↓	Not Sig	1-unit increase in AveOccup → 0.0126 decrease in house value
Longitude	-0.005341	Negative ↓	Not Sig	1-unit increase in Longitude → 0.0053 decrease in house value
HouseAge	0.000418	Positive ↑	Not Sig	1-unit increase in HouseAge → 0.0004 increase in house value
Population	0.000018	Positive ↑	Not Sig	1-unit increase in Population → 0.0000 increase in house value

OLS ASSUMPTION CHECKS (Critical Diagnostics)

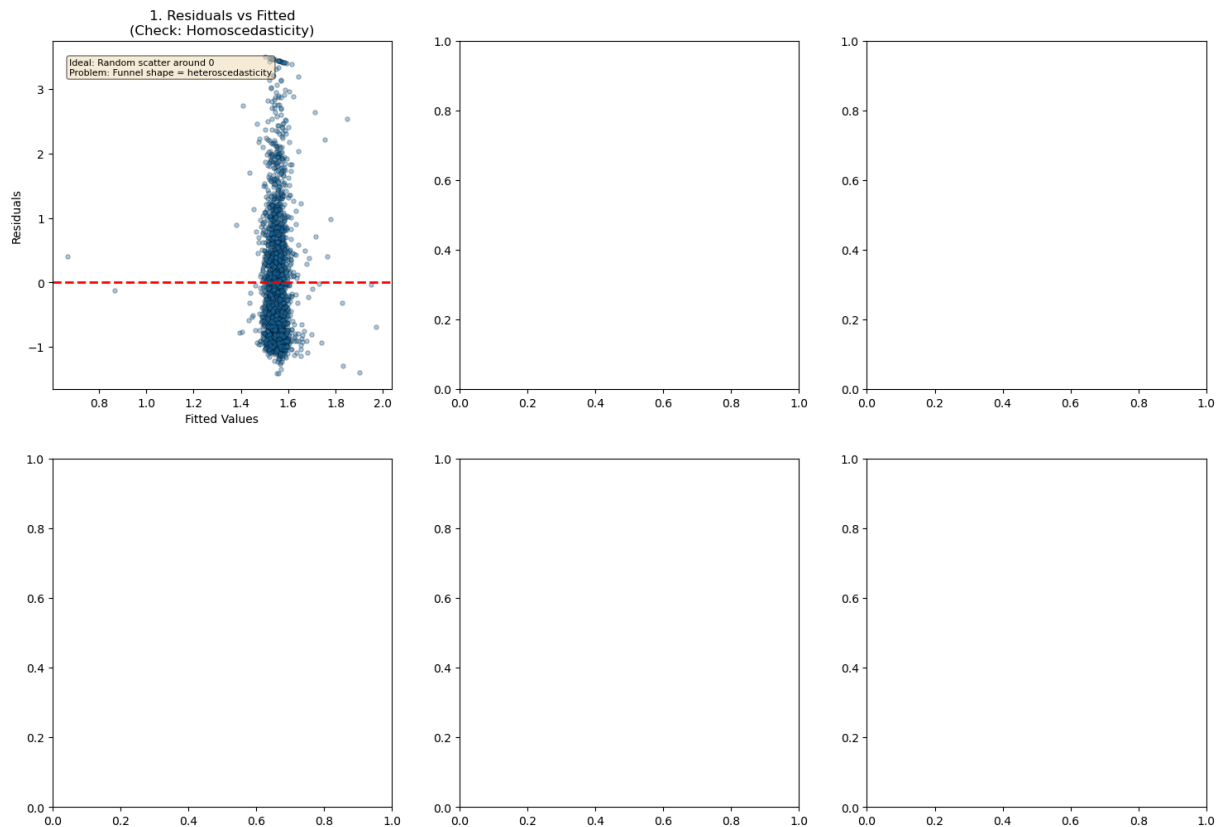
```
In [9]: predictions_train = ols_model.fittedvalues
        residuals = ols_model.resid

        fig, axes = plt.subplots(2, 3, figsize=(18, 12))
        fig.suptitle('OLS Regression Diagnostic Plots', fontsize=16, fontweight='bold')

        # --- Diagnostic 1: Residuals vs Fitted Values (Homoscedasticity) ---
        ax = axes[0, 0]
        ax.scatter(predictions_train, residuals, alpha=0.4, edgecolors='k', linewidth=0.5,
        ax.axhline(y=0, color='red', linestyle='--', linewidth=2)
        ax.set_xlabel('Fitted Values')
        ax.set_ylabel('Residuals')
        ax.set_title('1. Residuals vs Fitted\n(Check: Homoscedasticity)')
        ax.text(0.05, 0.95,
                'Ideal: Random scatter around 0\nProblem: Funnel shape = heteroscedasticity
                transform=ax.transAxes, fontsize=8, verticalalignment='top',
                bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

Out[9]: Text(0.05, 0.95, 'Ideal: Random scatter around 0\nProblem: Funnel shape = heterosc
edasticity')
```

OLS Regression Diagnostic Plots



```
In [10]: predictions_train = ols_model.fittedvalues
residuals = ols_model.resid

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('OLS Regression Diagnostic Plots', fontsize=16, fontweight='bold')

# --- Diagnostic 1: Residuals vs Fitted Values (Homoscedasticity) ---
ax = axes[0, 0]
ax.scatter(predictions_train, residuals, alpha=0.4, edgecolors='k', linewidth=0.5,
ax.axhline(y=0, color='red', linestyle='--', linewidth=2)
ax.set_xlabel('Fitted Values')
ax.set_ylabel('Residuals')
ax.set_title('1. Residuals vs Fitted\n(Check: Homoscedasticity)')
ax.text(0.05, 0.95,
        'Ideal: Random scatter around 0\nProblem: Funnel shape = heteroscedasticity',
        transform=ax.transAxes, fontsize=8, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# --- Diagnostic 2: Q-Q Plot (Normality of Residuals) ---
ax = axes[0, 1]
stats.probplot(residuals, dist="norm", plot=ax)
ax.set_title('2. Q-Q Plot\n(Check: Normality of Residuals)')
ax.text(0.05, 0.95,
        'Ideal: Points on diagonal line\nProblem: Curved tails = non-normal',
        transform=ax.transAxes, fontsize=8, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# --- Diagnostic 3: Histogram of Residuals ---
```



```

ax = axes[0, 2]
sns.histplot(residuals, kde=True, ax=ax, bins=40, color='steelblue')
ax.axvline(x=0, color='red', linestyle='--', linewidth=2)
ax.set_xlabel('Residual Value')
ax.set_ylabel('Frequency')
ax.set_title('3. Residual Distribution\n(Check: Bell-shaped / Normal)')

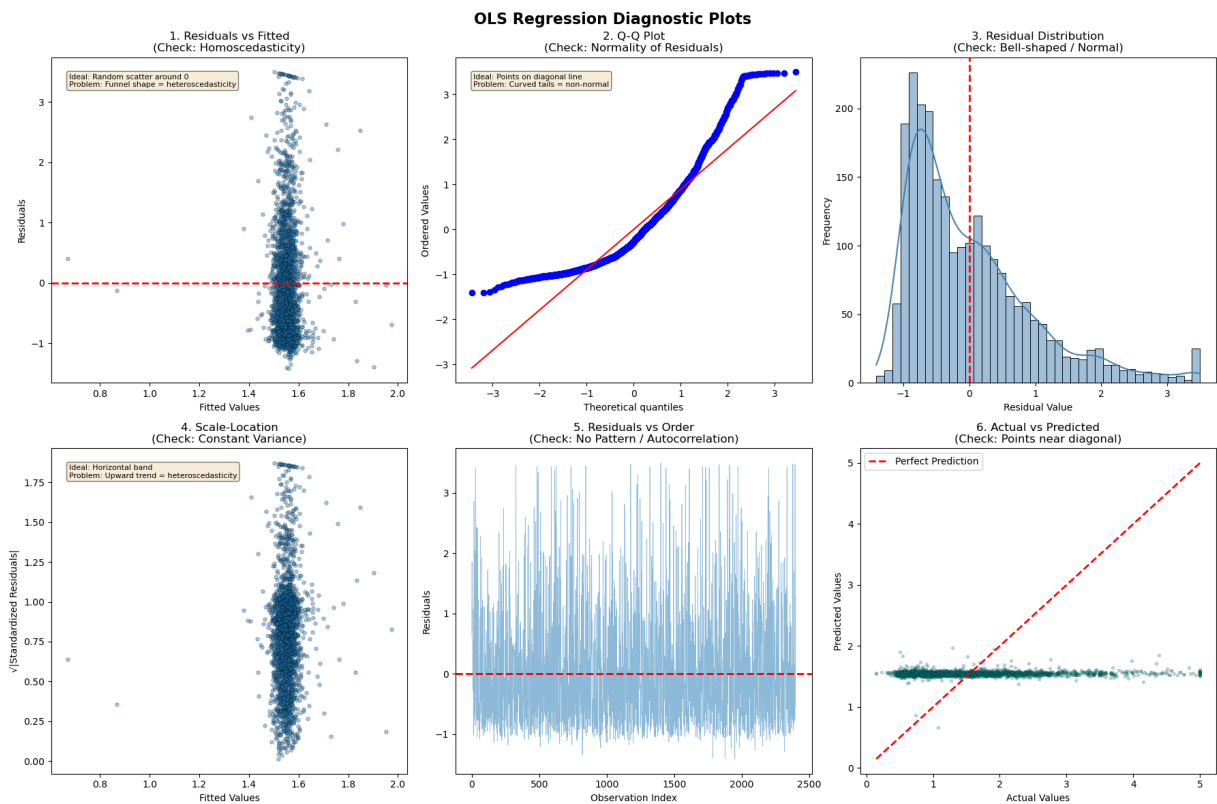
# --- Diagnostic 4: Scale-Location Plot (Homoscedasticity) ---
ax = axes[1, 0]
std_residuals = np.sqrt(np.abs(residuals))
ax.scatter(predictions_train, std_residuals, alpha=0.4, edgecolors='k', linewidth=0.3)
ax.set_xlabel('Fitted Values')
ax.set_ylabel('√|Standardized Residuals|')
ax.set_title('4. Scale-Location\n(Check: Constant Variance)')
ax.text(0.05, 0.95,
        'Ideal: Horizontal band\nProblem: Upward trend = heteroscedasticity',
        transform=ax.transAxes, fontsize=8, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# --- Diagnostic 5: Residuals vs Observation Order (Autocorrelation) ---
ax = axes[1, 1]
ax.plot(residuals.values, alpha=0.5, linewidth=0.5)
ax.axhline(y=0, color='red', linestyle='--', linewidth=2)
ax.set_xlabel('Observation Index')
ax.set_ylabel('Residuals')
ax.set_title('5. Residuals vs Order\n(Check: No Pattern / Autocorrelation)')

# --- Diagnostic 6: Actual vs Predicted ---
ax = axes[1, 2]
ax.scatter(y_train, predictions_train, alpha=0.3, edgecolors='k', linewidth=0.3, s=100)
min_val = min(y_train.min(), predictions_train.min())
max_val = max(y_train.max(), predictions_train.max())
ax.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='Perfect')
ax.set_xlabel('Actual Values')
ax.set_ylabel('Predicted Values')
ax.set_title('6. Actual vs Predicted\n(Check: Points near diagonal)')
ax.legend()

plt.tight_layout()
plt.show()

```



```
In [11]: # --- Formal Statistical Tests for Assumptions ---
print("\n--- Formal Statistical Tests ---")

# Test 1: Breusch-Pagan Test for Heteroscedasticity
bp_stat, bp_pvalue, bp_f, bp_fp = het_breuschpagan(residuals, X_train_sm)
print(f"\n1. Breusch-Pagan Test (Heteroscedasticity):")
print(f"    Statistic: {bp_stat:.4f}, p-value: {bp_pvalue:.2e}")
print(f"    → {'REJECT H0: Heteroscedasticity detected!' if bp_pvalue < 0.05 else 'Fail to reject H0: Homoscedasticity assumed.'}")

# Test 2: Shapiro-Wilk Test for Normality (sample up to 5000 for speed)
sample_size = min(len(residuals), 5000)
sw_stat, sw_pvalue = stats.shapiro(residuals.sample(sample_size, random_state=RANDOM_SEED))
print(f"\n2. Shapiro-Wilk Test (Normality of Residuals):")
print(f"    Statistic: {sw_stat:.6f}, p-value: {sw_pvalue:.2e}")
print(f"    → {'REJECT H0: Residuals are NOT normally distributed.' if sw_pvalue < 0.05 else 'Fail to reject H0: Residuals are normally distributed.'}")

--- Formal Statistical Tests ---
```

1. Breusch-Pagan Test (Heteroscedasticity):
Statistic: 9.2180, p-value: 3.24e-01
→ FAIL TO REJECT H0: Homoscedasticity (constant variance) assumed.
2. Shapiro-Wilk Test (Normality of Residuals):
Statistic: 0.880304, p-value: 1.39e-39
→ REJECT H0: Residuals are NOT normally distributed.

MULTICOLLINEARITY CHECK (Variance Inflation Factor)

```
In [12]: vif_data = pd.DataFrame()
vif_data['Feature'] = X_train.columns
vif_data['VIF'] = [variance_inflation_factor(X_train.values, i) for i in range(X_train.shape[1])]
```

```

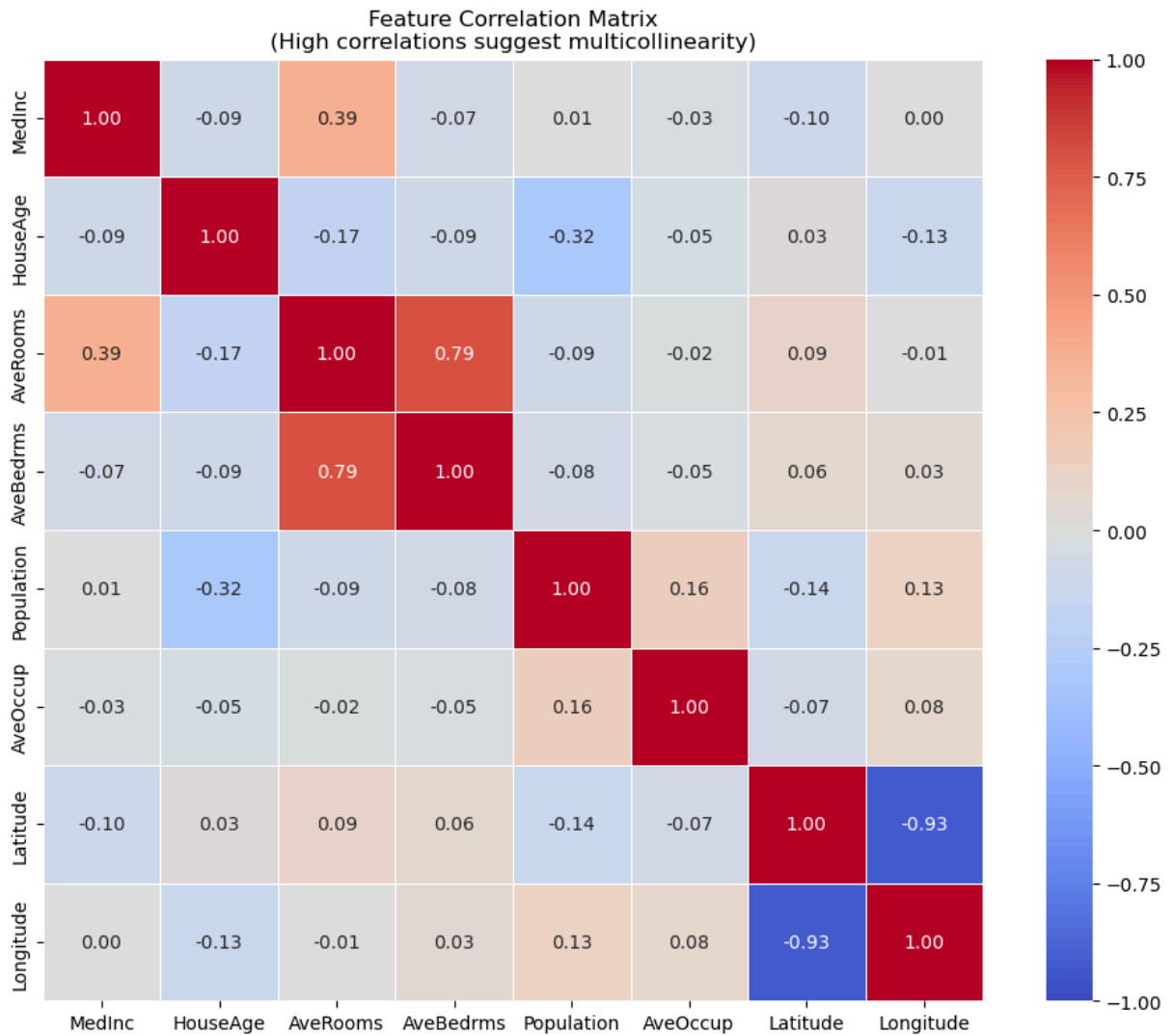
vif_data['Status'] = vif_data['VIF'].apply(
    lambda x: '⚠ HIGH' if x > 10 else ('⚡ MODERATE' if x > 5 else '✓ OK')
)
vif_data = vif_data.sort_values('VIF', ascending=False)

print(f"\n{'Feature':<12} | {'VIF':>10} | Status")
print("-" * 40)
for _, row in vif_data.iterrows():
    print(f"{row['Feature']:<12} | {row['VIF']:>10.4f} | {row['Status']}")

# Correlation Heatmap
plt.figure(figsize=(10, 8))
corr_matrix = X_train.corr()
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, vmin=-1, vmax=1, square=True, linewidths=0.5)
plt.title('Feature Correlation Matrix\n(High correlations suggest multicollinearity)')
plt.tight_layout()
plt.show()

```

Feature	VIF	Status
Longitude	661.1922	⚠ HIGH
Latitude	566.5703	⚠ HIGH
AveBedrms	48.8279	⚠ HIGH
AveRooms	47.0884	⚠ HIGH
MedInc	11.8873	⚠ HIGH
HouseAge	7.7481	⚡ MODERATE
Population	3.2306	✓ OK
AveOccup	2.9476	✓ OK



SKLEARN vs STATSMODELS COMPARISON (Prediction on Test Set)

```
In [13]: # Scikit-learn OLS (LinearRegression IS OLS under the hood)
sklearn_ols = LinearRegression()
sklearn_ols.fit(X_train, y_train)
y_pred_sklearn = sklearn_ols.predict(X_test)

# Statsmodels OLS prediction on test set
y_pred_sm = ols_model.predict(X_test_sm)

# Compare predictions (should be virtually identical)
print("\n--- Prediction Comparison (First 10 Test Samples) ---")
comparison = pd.DataFrame({
    'Actual': y_test.values[:10],
    'Statsmodels': y_pred_sm[:10],
    'Scikit-learn': y_pred_sklearn[:10],
    'Difference': np.abs(y_pred_sm[:10] - y_pred_sklearn[:10])
})
print(comparison.to_string(index=False, float_format=lambda x: f'{x:.6f}'))

# Performance Metrics
```

```

print("\n--- Test Set Performance Metrics ---")
metrics = {
    'Metric': ['MSE', 'RMSE', 'MAE', 'R²', 'Adjusted R²'],
    'Value': [
        mean_squared_error(y_test, y_pred_sklearn),
        np.sqrt(mean_squared_error(y_test, y_pred_sklearn)),
        mean_absolute_error(y_test, y_pred_sklearn),
        r2_score(y_test, y_pred_sklearn),
        1 - (1 - r2_score(y_test, y_pred_sklearn)) * (len(y_test) - 1) / (len(y_test) - 2)
    ]
}
metrics_df = pd.DataFrame(metrics)
print(metrics_df.to_string(index=False, float_format=lambda x: f'{x:.6f}'))

```

--- Prediction Comparison (First 10 Test Samples) ---

Actual	Statsmodels	Scikit-learn	Difference
0.940000	1.550602	1.550602	0.000000
0.962000	1.592546	1.592546	0.000000
2.534000	1.599821	1.599821	0.000000
0.806000	1.541120	1.541120	0.000000
0.905000	1.532599	1.532599	0.000000
0.934000	1.531750	1.531750	0.000000
2.332000	1.527755	1.527755	0.000000
0.598000	1.693883	1.693883	0.000000
3.629000	1.528307	1.528307	0.000000
1.430000	1.574966	1.574966	0.000000

--- Test Set Performance Metrics ---

Metric	Value
MSE	0.957292
RMSE	0.978413
MAE	0.784889
R²	-0.006791
Adjusted R²	-0.020419

INFLUENCE ANALYSIS (Cook's Distance)

```

In [14]: # Calculate Cook's Distance
influence = ols_model.get_influence()
cooks_distance, p_values = influence.cooks_distance

# Plot
plt.figure(figsize=(10, 5))
plt.stem(cooks_distance, markerfmt='', linefmt='b-', basefmt=' ')
plt.axhline(y=4/len(y_train), color='red', linestyle='--', label=f'Threshold (4/n =
plt.xlabel('Observation Index')
plt.ylabel('Cook's Distance')
plt.title("Cook's Distance (Influential Observations)")
plt.legend()
plt.tight_layout()
plt.show()

# Count influential points
threshold = 4 / len(y_train)
influential_count = (cooks_distance > threshold).sum()

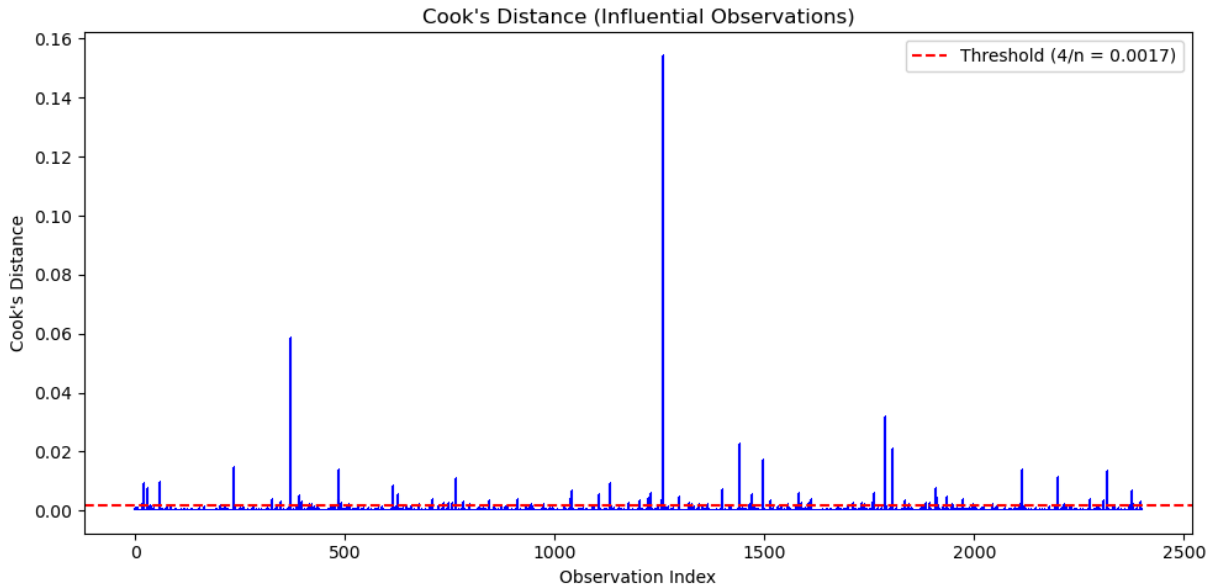
```

```

print(f"\nInfluential observations (Cook's D > 4/n): {influential_count} out of {le
print(f"Threshold: {threshold:.6f}")

if influential_count > 0:
    influential_idx = np.where(cooks_distance > threshold)[0][:5]
    print(f"\nTop influential observation indices: {influential_idx.tolist()}")
    print("Consider investigating these data points for errors or outliers.")

```



Influential observations (Cook's D > 4/n): 101 out of 2400
Threshold: 0.001667

Top influential observation indices: [16, 21, 29, 59, 202]
Consider investigating these data points for errors or outliers.

OLS EQUATION (Final Model Formula)

```

In [15]: print("\nŷ = ", end="")
terms = []
for feature, coef in zip(ols_model.params.index, ols_model.params.values):
    if feature == 'const':
        terms.append(f"{coef:.4f}")
    else:
        sign = "+" if coef >= 0 else "-"
        terms.append(f"{sign} {abs(coef):.4f}·{feature}")
print(" ".join(terms))

print("\n" + "=" * 80)
print("OLS REGRESSION COMPLETE")
print("=" * 80)

```

$\hat{y} = 1.4045 - 0.0139 \cdot \text{MedInc} + 0.0004 \cdot \text{HouseAge} + 0.0212 \cdot \text{AveRooms} - 0.0383 \cdot \text{AveBedrms} + 0.0000 \cdot \text{Population} - 0.0126 \cdot \text{AveOccup} - 0.0144 \cdot \text{Latitude} - 0.0053 \cdot \text{Longitude}$

=====
OLS REGRESSION COMPLETE
=====

REGRESSION APPROACH FOR MACHINE LEARNING

```
In [16]: import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
```

SETUP & TRAIN MODELS (Self-contained for easy copy-pasting)

```
In [17]: housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Train Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)

# Train Polynomial Regression
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)),
    ('lin_reg', LinearRegression())
])
poly_model.fit(X_train_scaled, y_train)

# Train Tree-Based Models
dt_model = DecisionTreeRegressor(max_depth=5, random_state=42)
dt_model.fit(X_train_scaled, y_train)

rf_model = RandomForestRegressor(n_estimators=50, random_state=42)
rf_model.fit(X_train_scaled, y_train)
```

Out[17]:

```
RandomForestRegressor ⓘ ⓘ
  Parameters
```

2. GENERATE TABLE OF COEFFICIENTS (Linear Models)

```

In [18]: print("="*80)
print("Note: These represent the exact mathematical weight of each feature.")
print("Positive coefficient = increases house price; Negative = decreases price.\n")

# --- Linear Regression Coefficients ---
lr_coefs = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_
})
# Add intercept
lr_intercept = pd.DataFrame({'Feature': ['INTERCEPT'], 'Coefficient': [lr_model.int
lr_table = pd.concat([lr_intercept, lr_coefs], ignore_index=True)

print("--- Linear Regression Coefficients ---")
print(lr_table.to_string(index=False))
print(f"\nBaseline/Intercept: {lr_model.intercept_:.4f}\n")

# --- Polynomial Regression Coefficients ---
# Get the expanded feature names from the polynomial step
poly_feature_names = poly_model.named_steps['poly'].get_feature_names_out(X.columns
poly_coefs = poly_model.named_steps['lin_reg'].coef_

poly_table = pd.DataFrame({
    'Polynomial Feature': poly_feature_names,
    'Coefficient': poly_coefs
})
# Add intercept
poly_intercept = pd.DataFrame({'Polynomial Feature': ['INTERCEPT'], 'Coefficient':
poly_table = pd.concat([poly_intercept, poly_table], ignore_index=True)

print("--- Polynomial Regression Coefficients (Top 10 by Absolute Value) ---")
# Sort by absolute value to show the most impactful polynomial terms
poly_table_sorted = poly_table.iloc[1:].copy() # Exclude intercept for sorting
poly_table_sorted['Abs_Coeff'] = poly_table_sorted['Coefficient'].abs()
poly_table_sorted = poly_table_sorted.sort_values(by='Abs_Coeff', ascending=False).h
poly_table_sorted = poly_table_sorted.drop('Abs_Coeff', axis=1)

# Re-add intercept at the top
poly_table_display = pd.concat([poly_intercept, poly_table_sorted], ignore_index=Tr
print(poly_table_display.to_string(index=False))
print(f"\nBaseline/Intercept: {poly_model.named_steps['lin_reg'].intercept_:.4f}\n")

```


=====

Note: These represent the exact mathematical weight of each feature.
Positive coefficient = increases house price; Negative = decreases price.

--- Linear Regression Coefficients ---

Feature	Coefficient
INTERCEPT	2.071947
MedInc	0.854383
HouseAge	0.122546
AveRooms	-0.294410
AveBedrms	0.339259
Population	-0.002308
AveOccup	-0.040829
Latitude	-0.896929
Longitude	-0.869842

Baseline/Intercept: 2.0719

--- Polynomial Regression Coefficients (Top 10 by Absolute Value) ---

Polynomial Feature	Coefficient
INTERCEPT	2.019034
AveOccup	-1.737660
Latitude	-1.161734
Longitude	-1.077405
MedInc	0.872411
AveRooms AveOccup	0.661246
AveBedrms AveOccup	-0.660779
AveOccup Latitude	0.576308
AveBedrms	0.533147
MedInc Latitude	-0.520628
HouseAge AveOccup	-0.478306

Baseline/Intercept: 2.0190

3. GENERATE TABLE OF FEATURE IMPORTANCES (Tree-Based Models)

```
In [19]: print("="*80)
print("Note: Tree models do not have linear coefficients. Instead, they use")
print("'Feature Importance' (0.0 to 1.0) to show how much each feature reduces")
print("impurity (variance) across all the decision trees.\n")

tree_importances = pd.DataFrame({
    'Feature': X.columns,
    'Decision Tree Importance': dt_model.feature_importances_,
    'Random Forest Importance': rf_model.feature_importances_
})

# Sort by Random Forest importance for readability
tree_importances = tree_importances.sort_values(by='Random Forest Importance', asce

print(tree_importances.to_string(index=False, float_format=lambda x: f'{x:.4f}'))
```

=====

Note: Tree models do not have linear coefficients. Instead, they use 'Feature Importance' (0.0 to 1.0) to show how much each feature reduces impurity (variance) across all the decision trees.

Feature	Decision Tree Importance	Random Forest Importance
MedInc	0.7712	0.5253
AveOccup	0.1284	0.1379
Latitude	0.0220	0.0888
Longitude	0.0021	0.0885
HouseAge	0.0416	0.0551
AveRooms	0.0313	0.0439
Population	0.0025	0.0310
AveBedrms	0.0009	0.0295

4. EXPLANATION FOR K-NNN

```
In [20]: print("="*80)
print("k-NN is an 'instance-based' or 'lazy' learning algorithm.")
print("It does NOT learn mathematical coefficients or feature importances.")
print("Instead, it memorizes the entire training dataset and makes predictions")
print("by calculating the distance (e.g., Euclidean) to the 'k' closest data points")
print("Therefore, no coefficient table can be generated for k-NN.")
print("="*80)
```

=====

k-NN is an 'instance-based' or 'lazy' learning algorithm.
It does NOT learn mathematical coefficients or feature importances.
Instead, it memorizes the entire training dataset and makes predictions
by calculating the distance (e.g., Euclidean) to the 'k' closest data points.
Therefore, no coefficient table can be generated for k-NN.

=====

STANDARD MACHINE LEARNING APPROACH FOR REGRESSION ANALYSIS

LOAD THE REQUIRED LIBRARIES

```
In [21]: import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
```

1. SETUP & TRAIN MODELS (Self-contained for easy copy-pasting)

```
In [22]: housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
In [23]: # Train Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train)
```

```
Out[23]: ▼ LinearRegression ⓘ ?
        ► Parameters
```

```
In [24]: # Train Polynomial Regression
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)),
    ('lin_reg', LinearRegression())
])
poly_model.fit(X_train_scaled, y_train)
```

```
Out[24]: ► Pipeline ⓘ ?
        ► PolynomialFeatures ⓘ
        │
        ► LinearRegression ⓘ
```

```
In [25]: # Train Tree-Based Models
dt_model = DecisionTreeRegressor(max_depth=5, random_state=42)
dt_model.fit(X_train_scaled, y_train)

rf_model = RandomForestRegressor(n_estimators=50, random_state=42)
rf_model.fit(X_train_scaled, y_train)
```

```
Out[25]: ▼ RandomForestRegressor ⓘ ?
        ► Parameters
```

2. GENERATE TABLE OF COEFFICIENTS (Linear Models)

TABLE 1: MATHEMATICAL COEFFICIENTS (Linear & Polynomial Models)

```
In [26]: print("="*80)
print("Note: These represent the exact mathematical weight of each feature.")
print("Positive coefficient = increases house price; Negative = decreases price.\n"
```

```
=====
Note: These represent the exact mathematical weight of each feature.
Positive coefficient = increases house price; Negative = decreases price.
```

```
In [27]: # --- Linear Regression Coefficients ---
lr_coefs = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lr_model.coef_
})
# Add intercept
lr_intercept = pd.DataFrame({'Feature': ['INTERCEPT'], 'Coefficient': [lr_model.int
lr_table = pd.concat([lr_intercept, lr_coefs], ignore_index=True)

print("--- Linear Regression Coefficients ---")
print(lr_table.to_string(index=False))
print(f"\nBaseline/Intercept: {lr_model.intercept_:.4f}\n")
```

```
--- Linear Regression Coefficients ---
```

Feature	Coefficient
INTERCEPT	2.071947
MedInc	0.854383
HouseAge	0.122546
AveRooms	-0.294410
AveBedrms	0.339259
Population	-0.002308
AveOccup	-0.040829
Latitude	-0.896929
Longitude	-0.869842

```
Baseline/Intercept: 2.0719
```

```
In [28]: # --- Polynomial Regression Coefficients ---
# Get the expanded feature names from the polynomial step
poly_feature_names = poly_model.named_steps['poly'].get_feature_names_out(X.columns
poly_coefs = poly_model.named_steps['lin_reg'].coef_

poly_table = pd.DataFrame({
    'Polynomial Feature': poly_feature_names,
    'Coefficient': poly_coefs
})
# Add intercept
poly_intercept = pd.DataFrame({'Polynomial Feature': ['INTERCEPT'], 'Coefficient':
poly_table = pd.concat([poly_intercept, poly_table], ignore_index=True)

print("--- Polynomial Regression Coefficients (Top 10 by Absolute Value) ---")
# Sort by absolute value to show the most impactful polynomial terms
poly_table_sorted = poly_table.iloc[1:].copy() # Exclude intercept for sorting
poly_table_sorted['Abs_Coef'] = poly_table_sorted['Coefficient'].abs()
poly_table_sorted = poly_table_sorted.sort_values(by='Abs_Coef', ascending=False).h
poly_table_sorted = poly_table_sorted.drop('Abs_Coef', axis=1)
```

```
# Re-add intercept at the top
poly_table_display = pd.concat([poly_intercept, poly_table_sorted], ignore_index=True)
print(poly_table_display.to_string(index=False))
print(f"\nBaseline/Intercept: {poly_model.named_steps['lin_reg'].intercept_:.4f}\n")

--- Polynomial Regression Coefficients (Top 10 by Absolute Value) ---
Polynomial Feature    Coefficient
INTERCEPT          2.019034
AveOccup              -1.737660
Latitude              -1.161734
Longitude              -1.077405
MedInc                 0.872411
AveRooms AveOccup      0.661246
AveBedrms AveOccup     -0.660779
AveOccup Latitude      0.576308
AveBedrms              0.533147
MedInc Latitude        -0.520628
HouseAge AveOccup      -0.478306

Baseline/Intercept: 2.0190
```

3. GENERATE TABLE OF FEATURE IMPORTANCES (Tree-Based Models)

```
In [29]: print("="*80)
print("TABLE 2: FEATURE IMPORTANCES (Tree-Based Models)")
print("="*80)
print("Note: Tree models do not have linear coefficients. Instead, they use")
print("'Feature Importance' (0.0 to 1.0) to show how much each feature reduces")
print("impurity (variance) across all the decision trees.\n")

tree_importances = pd.DataFrame({
    'Feature': X.columns,
    'Decision Tree Importance': dt_model.feature_importances_,
    'Random Forest Importance': rf_model.feature_importances_
})

# Sort by Random Forest importance for readability
tree_importances = tree_importances.sort_values(by='Random Forest Importance', ascending=False)

print(tree_importances.to_string(index=False, float_format=lambda x: f'{x:.4f}'))
```

=====

TABLE 2: FEATURE IMPORTANCES (Tree-Based Models)

=====

Note: Tree models do not have linear coefficients. Instead, they use 'Feature Importance' (0.0 to 1.0) to show how much each feature reduces impurity (variance) across all the decision trees.

Feature	Decision Tree Importance	Random Forest Importance
MedInc	0.7712	0.5253
AveOccup	0.1284	0.1379
Latitude	0.0220	0.0888
Longitude	0.0021	0.0885
HouseAge	0.0416	0.0551
AveRooms	0.0313	0.0439
Population	0.0025	0.0310
AveBedrms	0.0009	0.0295

4. EXPLANATION FOR K-NN

```
In [30]: print("\n" + "="*80)
print("NOTE ON K-NEAREST NEIGHBORS (k-NN)")
print("="*80)
print("k-NN is an 'instance-based' or 'lazy' learning algorithm.")
print("It does NOT learn mathematical coefficients or feature importances.")
print("Instead, it memorizes the entire training dataset and makes predictions")
print("by calculating the distance (e.g., Euclidean) to the 'k' closest data points")
print("Therefore, no coefficient table can be generated for k-NN.")
print("="*80)
```

=====

NOTE ON K-NEAREST NEIGHBORS (k-NN)

=====

k-NN is an 'instance-based' or 'lazy' learning algorithm.
It does NOT learn mathematical coefficients or feature importances.
Instead, it memorizes the entire training dataset and makes predictions
by calculating the distance (e.g., Euclidean) to the 'k' closest data points.
Therefore, no coefficient table can be generated for k-NN.

=====

Load the Required Libraries

```
In [31]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Scikit-Learn imports
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
```

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.feature_selection import SelectKBest, f_regression
from sklearn.pipeline import Pipeline

# XAI imports
import shap
import lime
import lime.lime_tabular

```

Set random seed for reproducibility

```

In [32]: np.random.seed(42)
RANDOM_STATE = 42

```

```

print("="*80)
print("PART 1: DATA LOADING & PREPARATION (Housing Price Prediction)")
print("="*80)

```

```

=====
PART 1: DATA LOADING & PREPARATION (Housing Price Prediction)
=====

```

```

In [33]: # 1. Load In-built Data
housing = fetch_california_housing()
housing

```

```

Out[33]: {'data': array([[ 8.3252, 41., 6.98412698, ..., 2.55555556,
    37.88, -122.23],
    [ 8.3014, 21., 6.23813708, ..., 2.10984183,
    37.86, -122.22],
    [ 7.2574, 52., 8.28813559, ..., 2.80225989,
    37.85, -122.24],
    ...,
    [ 1.7, 17., 5.20554273, ..., 2.3256351,
    39.43, -121.22],
    [ 1.8672, 18., 5.32951289, ..., 2.12320917,
    39.43, -121.32],
    [ 2.3886, 16., 5.25471698, ..., 2.61698113,
    39.37, -121.24]], shape=(20640, 8)),
'target': array([4.526, 3.585, 3.521, ..., 0.923, 0.847, 0.894], shape=(20640,)),
'frame': None,
'target_names': ['MedHouseVal'],
'feature_names': ['MedInc',
'HouseAge',
'AveRooms',
'AveBedrms',
'Population',
'AveOccup',
'Latitude',
'Longitude'],
'DESCR': '.. _california_housing_dataset:\n\nCalifornia Housing dataset\n-----
\n\n**Data Set Characteristics:**\n\n: Number of Instances: 20640
\n\n: Number of Attributes: 8 numeric, predictive attributes and the target\n\n: Att
ribute Information:\n    - MedInc            median income in block group\n    - House
Age            median house age in block group\n    - AveRooms            average number of ro
oms per household\n    - AveBedrms            average number of bedrooms per household\n
- Population    block group population\n    - AveOccup            average number of hous
ehold members\n    - Latitude            block group latitude\n    - Longitude            block
group longitude\n\n: Missing Attribute Values: None\n\nThis dataset was obtained fr
om the StatLib repository.\nhttps://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housin
g.html\n\nThe target variable is the median house value for California district
s,\nexpressed in hundreds of thousands of dollars ($100,000).\n\nThis dataset was
derived from the 1990 U.S. census, using one row per census\nblock group. A block
group is the smallest geographical unit for which the U.S.\nCensus Bureau publishe
s sample data (a block group typically has a population\nof 600 to 3,000 peopl
e).\n\nA household is a group of people residing within a home. Since the average
\nnumber of rooms and bedrooms in this dataset are provided per household, these\n
columns may take surprisingly large values for block groups with few households\na
nd many empty houses, such as vacation resorts.\n\nIt can be downloaded/loaded usi
ng the\n:func:`sklearn.datasets.fetch_california_housing` function.\n\n.. rubric::
References\n\n- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregression
s,\nStatistics and Probability Letters, 33:291-297, 1997.\n'}

```

```

In [34]: X = pd.DataFrame(housing.data, columns=housing.feature_names)
X.head()

```


Out[34]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25



In [35]: `y = pd.Series(housing.target, name='MedHouseVal') # Target: Median House Value`
`y.head()`

Out[35]:

0	4.526
1	3.585
2	3.521
3	3.413
4	3.422

Name: MedHouseVal, dtype: float64

In [36]: `# Subsample to 3000 rows for faster execution of complex models (GridSearch/LIME)`
`X = X.sample(n=3000, random_state=RANDOM_STATE)`
`y = y.loc[X.index]`

`print(f"Dataset shape: {X.shape}")`
`print(f"Target statistics:\n{y.describe()}\n")`

Dataset shape: (3000, 8)
Target statistics:
count 3000.000000
mean 2.071512
std 1.143336
min 0.175000
25% 1.213500
50% 1.814500
75% 2.659000
max 5.000010
Name: MedHouseVal, dtype: float64

In [37]: `# 2. Train-Test Split & Scaling`
`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

`scaler = StandardScaler()`
`X_train_scaled = scaler.fit_transform(X_train)`
`X_test_scaled = scaler.transform(X_test)`

PART 2: BASELINE REGRESSION MODELS & EVALUATION

2.1. Define Models

```
In [38]: reg_models = {
    'Linear Regression': LinearRegression(),
    'Polynomial Regression': Pipeline([
        ('poly', PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)),
        ('lin_reg', LinearRegression())
    ]),
    'k-NN Regressor': KNeighborsRegressor(n_neighbors=5),
    'Decision Tree': DecisionTreeRegressor(random_state=RANDOM_STATE),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=RANDOM_STATE)
}
```

2.2. Train and Evaluate

```
In [39]: print(f"{'Model':<25} | {'MSE':<10} | {'RMSE':<10} | {'MAE':<10} | {'R²':<10}")
print("-" * 75)

results = {}
for name, model in reg_models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    results[name] = {'MSE': mse, 'RMSE': rmse, 'MAE': mae, 'R2': r2}
print(f"{'name':<25} | {'mse':<10.4f} | {'rmse':<10.4f} | {'mae':<10.4f} | {'r2':<10.4f}")
```

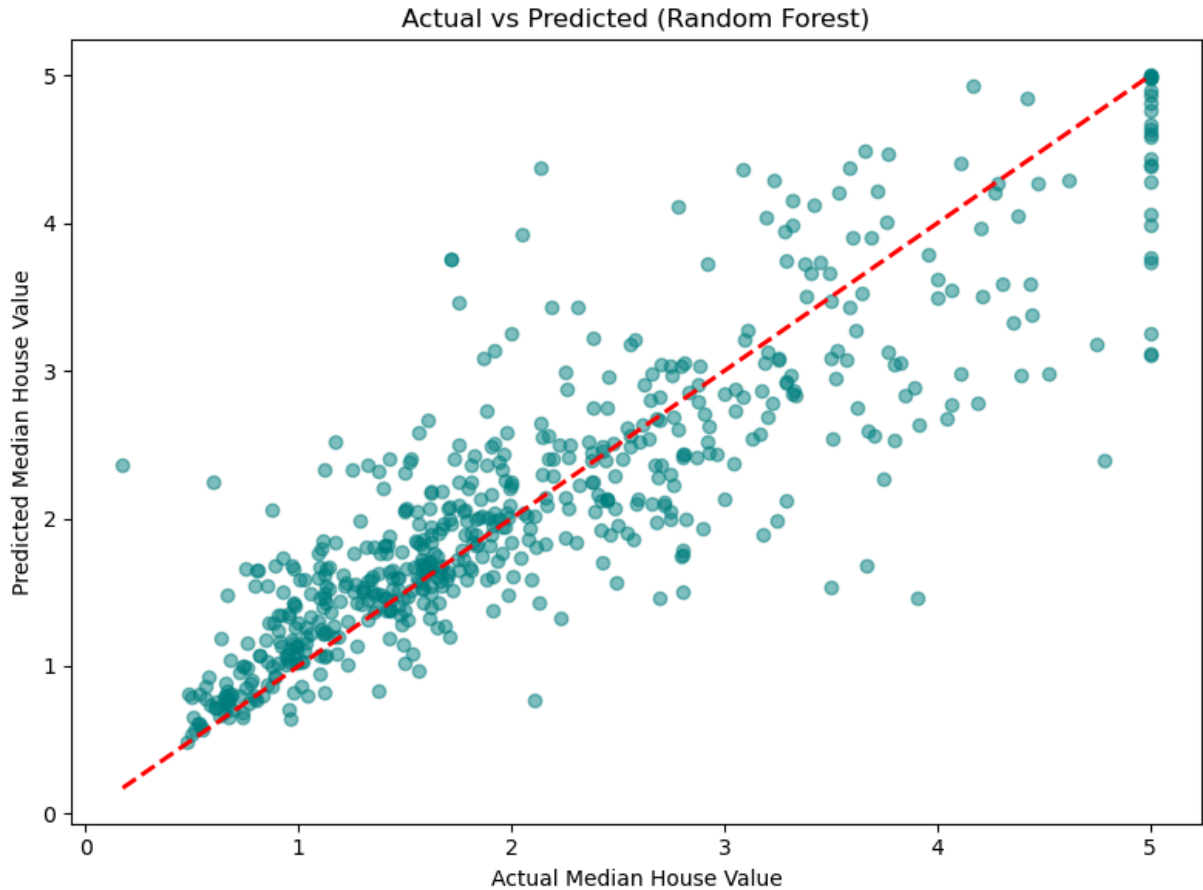
Model	MSE	RMSE	MAE	R²
Linear Regression	0.4887	0.6991	0.5245	0.6252
Polynomial Regression	0.4304	0.6560	0.4834	0.6700
k-NN Regressor	0.4250	0.6519	0.4633	0.6741
Decision Tree	0.6102	0.7811	0.5284	0.5321
Random Forest	0.3297	0.5742	0.3984	0.7472

2.3. Plot Actual vs Predicted for the best baseline model (Random Forest)

```
In [40]: best_baseline_name = 'Random Forest'
y_pred_best = reg_models[best_baseline_name].predict(X_test_scaled)

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_best, alpha=0.5, color='teal')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual Median House Value')
plt.ylabel('Predicted Median House Value')
plt.title(f'Actual vs Predicted ({best_baseline_name})')
```

```
plt.tight_layout()
plt.show()
```



PART 3: MODEL OPTIMIZATION (Bias-Variance, Tuning, Feature Selection)

3.1. Bias-Variance Tradeoff (Overfitting vs Underfitting)

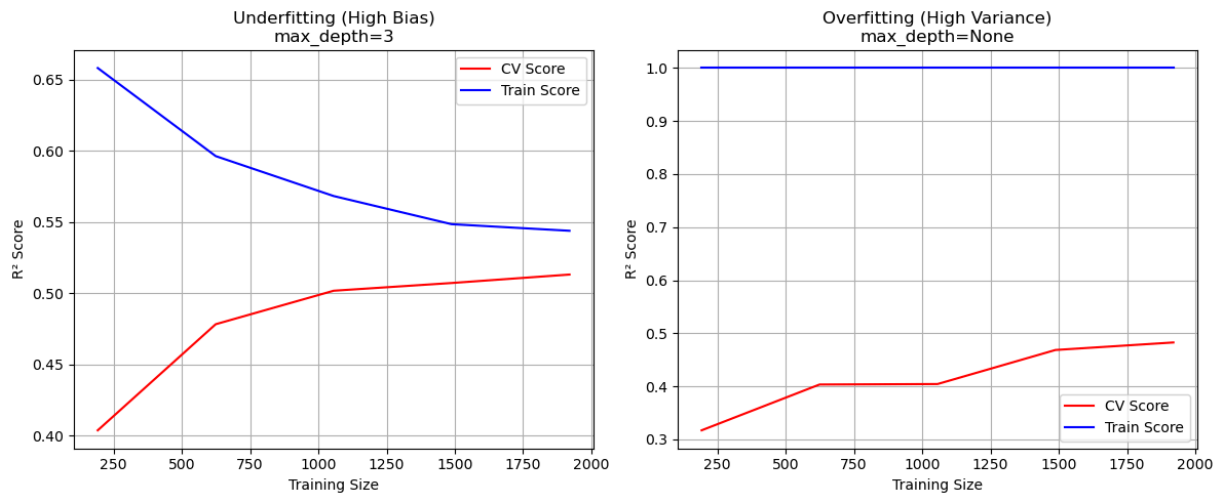
```
In [41]: print("\n--- Visualizing Bias-Variance Tradeoff via Learning Curves ---")
# Underfitting (High Bias): Shallow tree
dt_under = DecisionTreeRegressor(max_depth=3, random_state=RANDOM_STATE)
# Overfitting (High Variance): Unrestricted deep tree
dt_over = DecisionTreeRegressor(max_depth=None, random_state=RANDOM_STATE)

train_sizes, train_scores_under, test_scores_under = learning_curve(
    dt_under, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=-1)
train_sizes, train_scores_over, test_scores_over = learning_curve(
    dt_over, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=-1)

plt.figure(figsize=(12, 5))
# Plot Underfitting
plt.subplot(1, 2, 1)
plt.plot(train_sizes, np.mean(test_scores_under, axis=1), label='CV Score', color='blue')
plt.plot(train_sizes, np.mean(train_scores_under, axis=1), label='Train Score', color='red')
plt.title('Underfitting (High Bias)\nmax_depth=3')
plt.xlabel('Training Size'); plt.ylabel('R² Score'); plt.legend(); plt.grid(True)
```

```
# Plot Overfitting
plt.subplot(1, 2, 2)
plt.plot(train_sizes, np.mean(test_scores_over, axis=1), label='CV Score', color='r')
plt.plot(train_sizes, np.mean(train_scores_over, axis=1), label='Train Score', color='b')
plt.title('Overfitting (High Variance)\nmax_depth=None')
plt.xlabel('Training Size'); plt.ylabel('R² Score'); plt.legend(); plt.grid(True)
plt.tight_layout()
plt.show()
```

--- Visualizing Bias-Variance Tradeoff via Learning Curves ---



3.2. Feature Selection

```
In [42]: print("\n--- Feature Selection (SelectKBest) ---")
selector = SelectKBest(score_func=f_regression, k=4)
X_train_selected = selector.fit_transform(X_train_scaled, y_train)
X_test_selected = selector.transform(X_test_scaled)

selected_features = X.columns[selector.get_support()].tolist()
print(f"Top 4 Selected Features: {selected_features}")

# Retrain Random Forest on selected features
rf_selected = RandomForestRegressor(n_estimators=100, random_state=RANDOM_STATE)
rf_selected.fit(X_train_selected, y_train)
y_pred_sel = rf_selected.predict(X_test_selected)
print(f"Random Forest R² with Top 4 Features: {r2_score(y_test, y_pred_sel):.4f}")
```

--- Feature Selection (SelectKBest) ---

Top 4 Selected Features: ['MedInc', 'HouseAge', 'AveRooms', 'Latitude']

Random Forest R² with Top 4 Features: 0.6012

3.3. Hyperparameter Tuning

```
In [43]: # GridSearchCV for Random Forest
print("\n--- Hyperparameter Tuning: GridSearchCV (Random Forest) ---")
rf_grid = RandomForestRegressor(random_state=RANDOM_STATE)
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [10, 20, None],
```

```

    'min_samples_split': [2, 5]
}
grid_search = GridSearchCV(rf_grid, param_grid, cv=3, scoring='r2', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)
print(f"Best GridSearch Parameters: {grid_search.best_params_}")
print(f"Best GridSearch R2: {grid_search.best_score_:.4f}")

# RandomizedSearchCV for k-NN
print("\n--- Hyperparameter Tuning: RandomizedSearchCV (k-NN) ---")
knn_rand = KNeighborsRegressor()
param_dist = {
    'n_neighbors': np.arange(3, 20),
    'weights': ['uniform', 'distance'],
    'p': [1, 2] # 1 for Manhattan, 2 for Euclidean
}
rand_search = RandomizedSearchCV(knn_rand, param_dist, n_iter=15, cv=3,
                                scoring='neg_mean_squared_error', n_jobs=-1, random_state=42)
rand_search.fit(X_train_scaled, y_train)
print(f"Best RandomSearch Parameters: {rand_search.best_params_}")
print(f"Best RandomSearch MSE: {-rand_search.best_score_:.4f}")

--- Hyperparameter Tuning: GridSearchCV (Random Forest) ---
Best GridSearch Parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}
Best GridSearch R2: 0.7330

--- Hyperparameter Tuning: RandomizedSearchCV (k-NN) ---
Best RandomSearch Parameters: {'weights': 'distance', 'p': 1, 'n_neighbors': np.int64(13)}
Best RandomSearch MSE: 0.4102

```

PART 4: EXPLAINABLE AI (XAI) FOR REGRESSION

4.1. Use the best tuned Random Forest for explanations

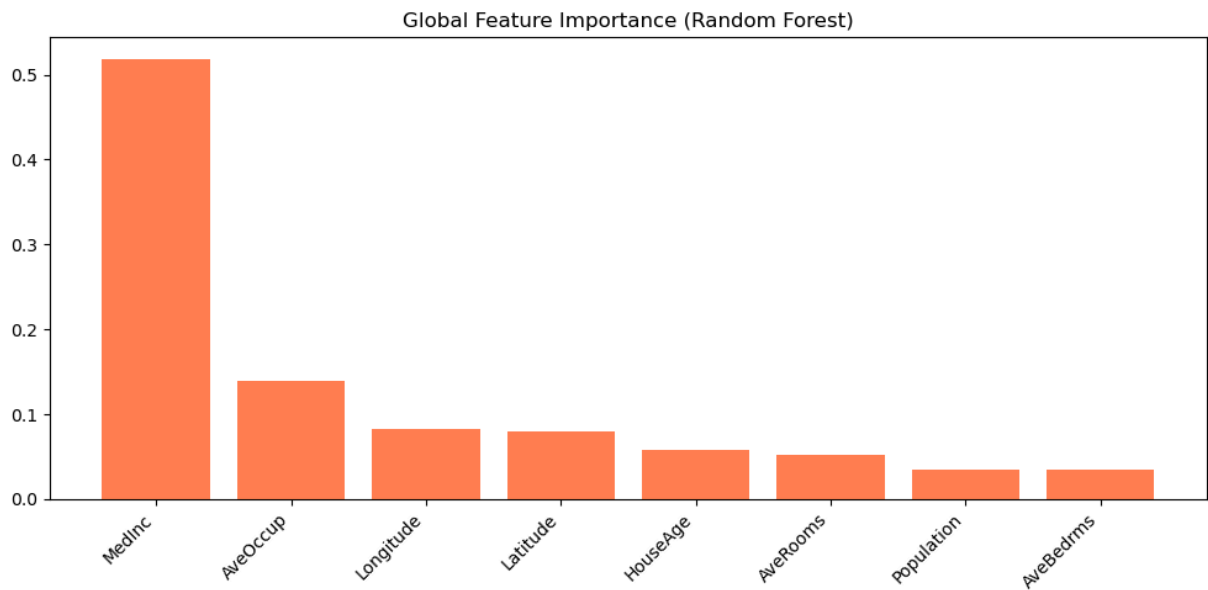
```

In [44]: best_rf = grid_search.best_estimator_
best_rf.fit(X_train_scaled, y_train) # Refit on full training data

# 1. Global Feature Importance (Built-in)
importances = best_rf.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10, 5))
plt.title("Global Feature Importance (Random Forest)")
plt.bar(range(X.shape[1]), importances[indices], align="center", color='coral')
plt.xticks(range(X.shape[1]), [X.columns[i] for i in indices], rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

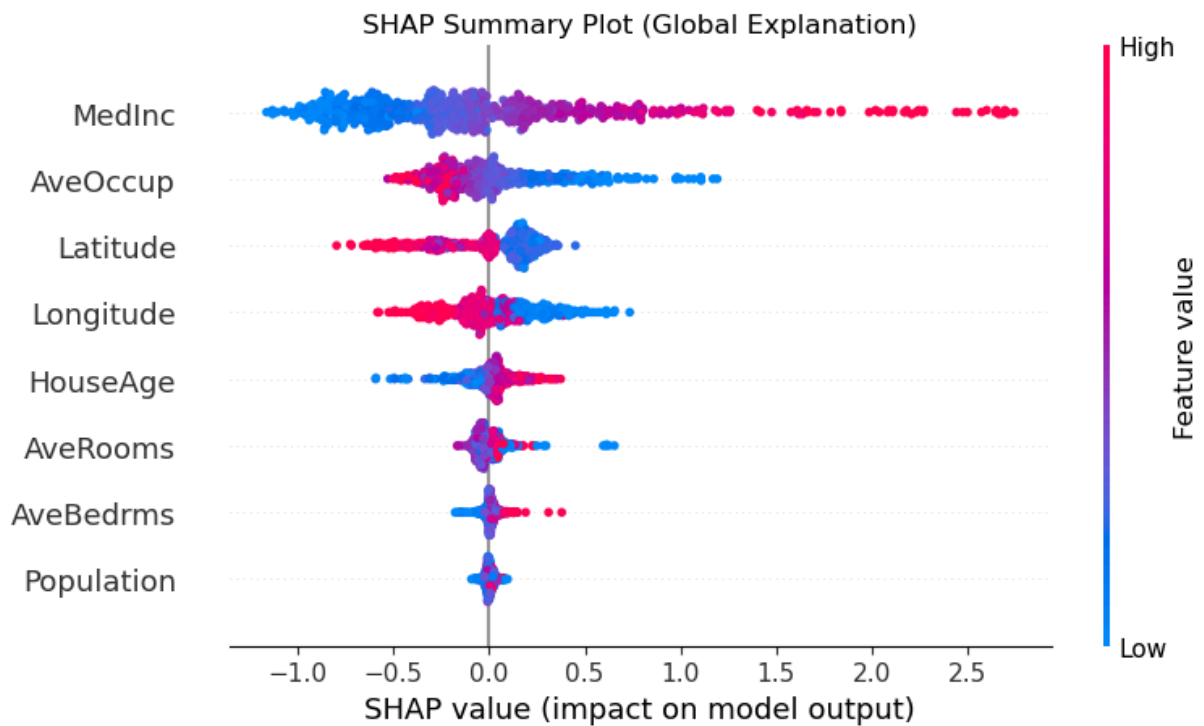


4.2. SHAP (Global & Local Explanations)

```
In [45]: print("\n--- SHAP Analysis ---")
# TreeExplainer is highly efficient for tree-based models
shap_explainer = shap.TreeExplainer(best_rf)
shap_values = shap_explainer.shap_values(X_test_scaled)

# Global Explanation: Summary Plot
plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, X_test_scaled, feature_names=X.columns, show=False)
plt.title("SHAP Summary Plot (Global Explanation)")
plt.tight_layout()
plt.show()
```

--- SHAP Analysis ---



4.3. LIME (Local Explanation)

```
In [46]: print("\n--- LIME Analysis (Local Explanation for Instance #0) ---")
lime_explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train_scaled,
    feature_names=X.columns,
    mode='regression'
)

# Explain the first instance in the test set
instance_idx = 0
exp = lime_explainer.explain_instance(
    X_test_scaled[instance_idx],
    best_rf.predict,
    num_features=5
)

print(f"Actual Value: {y_test.iloc[instance_idx]:.4f}")
print(f"Predicted Value: {best_rf.predict(X_test_scaled[instance_idx]).reshape(1, -1)}")
print("\nLIME Local Feature Contributions:")
for feature, weight in exp.as_list():
    print(f" {feature:<30} : {weight:+.4f}")

# Note: exp.show_in_notebook() generates an HTML widget if running in Jupyter.
# In a standard script, we print the list as done above.

print("\n" + "="*80)
print("Session 6 (Regression Only) Complete!")
print("="*80)
```

--- LIME Analysis (Local Explanation for Instance #0) ---

Actual Value: 0.9450
Predicted Value: 1.0542

LIME Local Feature Contributions:

MedInc <= -0.68	:	-1.0609
AveOccup > 0.14	:	-0.3748
-0.78 < Latitude <= -0.64	:	+0.3313
0.53 < Longitude <= 0.76	:	-0.1131
AveBedrms <= -0.24	:	-0.0933

=====
Session 6 (Regression Only) Complete!
=====